Pulse CBD — Project Report and Handover

As of: Wednesday 4 November 2026, 8 pm (Sydney)
Showcase: Friday 6 November 2026
Feature freeze: 30 October 2026 (the background map, 4 Nov, was added after it at the owner's request)
Author: Yoseph Kim, with Claude Code
Purpose: a complete record of the project — what it is, how it works, why every choice was made, what's left, and the exact state of the machine and repository — so that a new Claude Code session (or a teammate) can carry on from this document alone.

---

0. Read this first (for a new Claude session)

Where things are

Thing	Location
Project folder	`~/pulse-cbd` (Python 3.13 venv in `.venv`; activate with `source .venv/bin/activate`)
Current branch	`w12merge` = `yosephW12` + the teammate's `integration` work (latest commit: `git log -1`), pushed to GitHub. This is the build the Mac runs
GitHub remote	`group-repo` → `https://github.com/0rangeGoose/Civil3704-group-6` (private)
Decision log (newest first, every non-obvious call and its reasoning)	`docs/DECISIONS.md`
How to run everything	`README.md`
This report	`docs/PROJECT_REPORT.md` and `.pdf`
Original project brief	`~/Downloads/pulse-cbd-claude-code-prompt.md`
Teammates' original dashboard (separate codebase)	branches `mid-project-demo`, `test-pipline` on GitHub

What is running on the Mac right now (see §12 for details)

- The Pulse CBD **server** (`scripts/run_publish.py`), kept running by the **launchd job** `com.pulsecbd.server`: it starts at login and restarts within 30 s if it stops (`kill` only restarts it; stop with `python scripts/install_server_service.py --uninstall`). It serves `http://127.0.0.1:8765/` (projector view) and `/control` (tablet), polls TfNSW live and **records continuously**. Log: `data/recordings/server.log`.
- A **launchd job** `com.pulsecbd.recorder` runs `scripts/run_record.py` every weekday at 06:55 and 16:25 for 130 minutes. It stands aside while the server is polling.
- **Sleep is disabled** (`sudo pmset -a disablesleep 1`). Undo after the showcase with `sudo pmset -a disablesleep 0`.

The owner's working preferences (from `~/.claude/CLAUDE.md`)

- Ask for confirmation before any git commit unless told to commit; never push to a branch they didn't name. The latest instruction: **push new work to `yosephW12`, not `integration`.**
- Use the `gh` CLI for GitHub.

- When instructions are unclear or conflicting, stop and ask; never override or skip an instruction without asking.
- Explain things in plain language; the owner asks "what do you mean by…" when jargon slips in.

First things to check in a new session

```
cd ~/pulse-cbd && git status -sb && git log --oneline -3
lsof -nP -iTCP:8765 -sTCP:LISTEN          # is the server running?
curl -s http://127.0.0.1:8765/health       # vehicles, feed age, API quota use, playback mode
tail -5 data/recordings/recorder.log       # scheduled peak recordings
source .venv/bin/activate && pytest -q     # 193 tests should pass
```

Open decisions waiting on the owner are listed in §16; **remaining work** in §14.

---

1. Summary and status

Pulse CBD projects Sydney's live public transport network onto a physical scale model of the inner city. Every train, metro, light rail vehicle, ferry and bus in the model's footprint (typically 270–450 at once) appears as a moving dot on its route. Layers on top answer planning questions: where buses are held up, where they are crowded, where many jobs meet little public transport, and how peaks compare. It runs live, or replays recorded peaks as a time-lapse that can be paused and scrubbed.

Status at the feature freeze:

Plan step (agreed 25 Sep)	Status
1. Footprint, projector view, tablet controller, scheduled recording	Done
2. TfNSW traffic and crowding on 200 m street segments	Done
3. Replay and time-lapse	Done, plus pause and scrub (30 Oct)
4. Jobs, service gap, Opal 2016 on travel zones and a 150 m grid	Done
5. Combined "pressure" score with adjustable weights	Done
6. Our own congestion measure + agreement with TfNSW	Done (method changed; result is weak agreement — see §8)
7. Model testing, report figures, rehearsal	Not started — needs the physical model
Extras	BOAM bus-load layers; quota-aware polling; timetable refreshed; teammate's stop history, route filter and arrows (merged 30 Oct); optional background map (4 Nov)

- **Tests:** 193 automated tests pass in about 2 seconds, no network needed.
- **Data recorded:** 15 weekday peaks so far (14 complete, 1 partial; Tue 3 Nov PM lost — see §12), plus nights, middays and weekends.
- **Biggest risks now:** nothing has been tested on the physical model yet; the merged build (`w12merge`) has had little use; showcase networking (a Mac can't host a hotspot and join Wi-Fi at once); Wi-Fi drops during peaks.

---

2. Context

2.1 The course and team

CIVIL3704 group project (Group 6). Final showcase **6 Nov 2026**, where the team presents to assessors and visitors try the installation. Team roles aren't formally defined. The owner (Yoseph) built `pulse-cbd`; teammate **0rangeGoose** built a separate dashboard and has since been pushing to `integration` (§11).

2.2 The physical setup

- **Model:** a tabletop scale model of Sydney's inner city. Its footprint was surveyed by the team: **NE –33.835123, 151.275993; SW –33.894723, 151.132966** — about **13.2 km east–west × 6.6 km north–south (≈2:1)**. These come from the team's `MODEL_CORNERS` and are in `config/config.yaml` as `bounding_box`.
- **Projector:** above the model, **mirroring the laptop screen**. Anything visible on the laptop — buttons, cursor, notifications — lands on the model. This drove the whole control design (§7).
- **Access:** the rig stays set up between **weekly** sessions.
- **Users:** the team presents; visitors then use a tablet next to the model.

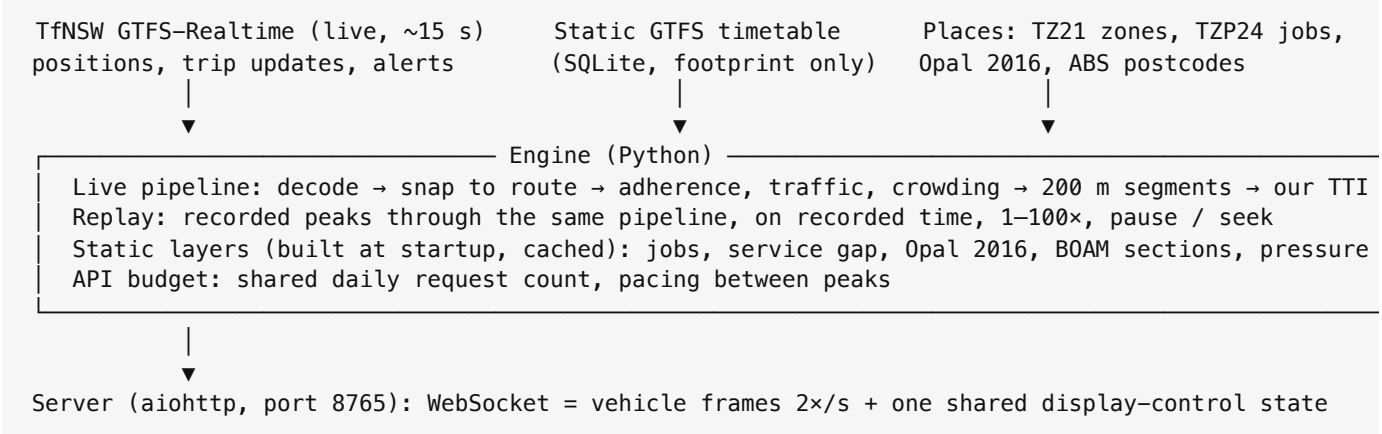
2.3 How the plan was set (25 Sep "grilling" session)

The owner and Claude worked through the design question by question. Key outcomes, all recorded in `docs/DECISIONS.md` (25 Sep entry):

- `pulse-cbd` (the owner's prototype) is the single codebase. Take the teammates' **planning-tool direction** (congestion, demand, peak comparison) but **rebuild their calculations**, which had flaws (§7.1).
- Footprint from the team's surveyed corners; the old 5 × 4 km box was a placeholder.
- Projection shows only map content; controls on a tablet (visitor view, PIN-locked team panel, stats) on the laptop's hotspot, keyboard as a backup.
- Congestion: TfNSW's own label first, then our own measure validated against it (step 6).
- Demand: live crowding, jobs, service gap and Opal 2016, each toggleable, on zones *and* a grid, plus a combined pressure layer.
- Replay recorded AM/PM peaks, time-lapse as the showcase default; record weekday peaks automatically.
- Code local-only at first (later pushed — §11).

---

3. How it works



```
                                HTTP = /areas /boam /pressure /segments /recordings /health /calibrati
            │                                                ▲
            ▼                                                │ control patches
Projector view (laptop, full-screen, mirrored)      Tablet controller (/control on the laptop's hot
```

- **One source of truth for display state.** Every button (tablet or laptop keyboard) sends a small patch to the server; the server validates it (`engine/publish/control.py`) and broadcasts the merged state to every page. The tablet and projector can never disagree.
- **Replay uses the live engine.** Recorded payloads go through exactly the same code, and the engine treats each payload's fetch time as "now", so 10-minute windows mean 10 *recorded* minutes even at 25×.
- **Static layers are computed once** at startup in the background and cached until an input file changes, so startup isn't delayed.

---

4. Data sources

Dataset	What it gives	Coverage	How obtained	Caveats
TfNSW GTFS-Realtime	Vehicle positions, `congestion_level`, `occupancy_status`, per-carriage crowding (1007 extension), trip-update delays, alerts	7 modes, ~15 s	Live, our API key (`.env`, never committed)	Traffic level only for buses and Metro; Sydney Trains crowding mostly per carriage; light rail and ferries report neither
TfNSW static GTFS	Routes, stops, trips, stop times, shapes	Refreshed 30 Oct: calendar **30 Oct 2026 – 28 Jan 2027**; 546 routes, 2,905 stops, 3,135 shapes in the footprint	Complete GTFS bundle via API, filtered to SQLite	Old 19 Aug copy kept as `gtfs_cbd.2026-08-19.db`
Travel Zones 2021 (TZ21)	Zone boundaries	405 zones in the footprint	From the team branch via Git LFS	Zones vary greatly in size
TZP24 employment	Jobs by place of work per zone	`EMP_2026` used	Team branch via Git LFS	A projection, not a count
Opal Tap On/Off Release 2	Taps in 15-min bins	21–27 Nov 2016; stations by name, buses by postcode	data.nsw.gov.au	Pre-Metro, pre-CBD light rail
ABS Postal Areas 2021	Postcode boundaries	40 postcodes touch the footprint	ABS shapefile, cut to footprint	Approximate postcodes
TfNSW BOAM	Every bus trip at every stop: scheduled/actual arrival, capacity, occupancy band	13 weekdays used (28–30 Sep holidays + 12–16, 19–23 Oct term) + 4 weekend days kept	Open Data Hub (login), `scripts/import_boam.py`	"Actual" times are GTFS-R predictions since Mar 2026; non-tappers uncounted
Our recordings	Raw live payloads, trimmed to a 10%-padded footprint	Continuous since 24 Oct; ~3.5 GB/day	Server + scheduled recorder	Gaps when Wi-Fi drops; Aug–Sep recordings deleted (rate-limited, unusable) except

Dataset	What it gives	Coverage	How obtained	Caveats
				weekends 22 Aug, 6 Sep

TfNSW API quota (measured, not documented): about **60,000 requests per day per key, resetting at midnight Sydney time.** On 25 Oct the key was refused after 59,964 requests counted from midnight. Polling all 15 endpoints every 15 s would cost ~86,000/day, so pacing is automatic (§10.3).

---

5. Features

5.1 On the projection

- **Vehicles** (always on): snapped to their route, smoothly interpolated between updates, short comet tails. Size shows mode (rail larger). Five mode groups can be hidden. Colour follows the chosen **lens**: schedule adherence (default), TfNSW traffic, or TfNSW crowding. Dots turn **white** while an area layer is on.
- **Interchange pulses:** nine major stations glow brighter as more services pass within 300 m.
- **Street layers** (one at a time):

Layer	Source	Time basis
Traffic	TfNSW `congestion_level`, buses	Live, last 10 min
Traffic (our measure)	Travel-time index vs night free flow (§6.10)	Live, last 10 min
Crowding	TfNSW occupancy	Live, last 10 min
Typical bus load	BOAM, share of capacity	Typical weekday AM or PM peak
Where buses lose time	BOAM, delay gained per 100 m	Typical weekday AM or PM peak

- **Area layers** (one at a time, on travel zones or the 150 m grid): Jobs, Service gap, Opal 2016, Pressure (combined, live).
- **Stop and route layers** (teammate, merged 30 Oct): stop markers; **stop history** (median or average delay, or crowding, per stop, from live stop visits; a stop needs ≥ 20 visits to be coloured); **route filter** singling out up to 8 routes with their stops in fixed colours. Vehicles are drawn as **arrows** showing direction.
- **Background map** (4 Nov): Plain (default), Detailed (OpenStreetMap roads, rail, water, parks) or Full (Detailed plus suburb, station, landmark, park, water and street names, never overlapping). Stored offline; see `DECISIONS.md` (4 Nov).
- **Projection hygiene:** no UI or cursor; everything clipped to the footprint; plain background by default (the model has the streets). An optional status line can be turned on from the team panel.

5.2 Time

- Live by default.
- Replay any recorded AM peak (7:00–9:00) or PM peak (16:30–18:30) with ≥10% of expected bus updates, or the latest two hours. Speeds 1×, 10×, 25× (default, a 2-hour peak in under 5 minutes), 50×. Replays loop.
- **Pause / play** (tablet or Space): the projection freezes and resumes without jumps.
- **Scrub:** tap anywhere on the timeline, ±5 min buttons, ←/→ for one minute. A jump first feeds the 10 recorded minutes before the target unpaced, so street layers are already filled.

- **Live polling and the server's recording pause during any replay** — always switch back to Live after a demo.

5.3 The tablet (`/control`)

- **Display (visitors):** time and replay controls, vehicle modes, dot colour, street layer, stop markers and stop history, route filter, area layer, area shape, BOAM peak (AM/PM), background map, and a legend explaining whatever is on.
- **Team (PIN `3704`, browser-only check):** theme, verification overlay, status line on projection, alignment nudge (2 px / 20 px, scale), crosshair calibration, pressure weights (0–3), system health (vehicles per mode, feed age, API requests used today and current polling interval).
- **Stats:** on-time % by mode over 5/15/60 min, active alerts, busiest interchanges.

5.4 Laptop keyboard (backup; every key goes through the server)

C calibrate · V verification overlay · H status line · T theme · M background map · P pulses · S stop markers · O stop history · L dot colour · K street layer · A area layer · G zones/grid · B BOAM peak · Space pause/play · 1–5 mode groups · arrows nudge (Shift ×10) · [ ] scale · 0 reset nudge.

---

6. How everything is measured

6.1 Map matching (`engine/normalise/map_match.py`)

A vehicle's GPS point is snapped onto its trip's route shape; if the trip isn't in the timetable, onto the nearest shape within 100 m (this is what places Sydney Trains, Metro and CBD light rail, whose realtime IDs differ from the timetable's). The search index holds 32-point pieces of each shape — whole shapes made the lookup ~14 ms per vehicle and most of the CPU.

6.2 Schedule adherence (`engine/normalise/adherence.py`)

Delay = TfNSW's predicted delay at the vehicle's **next stop** (trip updates). Threshold T: 300 s suburban rail, buses, ferries; 360 s intercity; 120 s Metro and light rail. On time ≤ T; late ≤ $2T$; very late > $2T$; cancelled; unknown. **Origin of *T*:** rail/metro/light rail values come from the original project brief as "sensible defaults" resembling TfNSW's published standards (not verified against an official source); buses and ferries were set to 300 s by analogy; the 2× "very late" cutoff is our default. Early running counts as on time. Report these as *the project's simplified thresholds*.

6.3 Interchange activity (`engine/normalise/aggregate.py`)

Distinct vehicles within 300 m of each of nine interchanges over a rolling window; glow scales with √count ÷ 5, capped at 1.

6.4 TfNSW traffic and crowding levels (`engine/normalise/levels.py`)

Level	Traffic (`congestion_level`)	Crowding (`occupancy_status`)
0	Running smoothly	Empty / many seats
1	Stop and go	Few seats
2	Congestion	Standing room only

Level	Traffic (`congestion_level`)	Crowding (`occupancy_status`)
3	Severe	Crushed / full / not accepting

Traffic counts **buses only** (Metro's level isn't road traffic). Trains without a vehicle-level crowding value use the **median carriage**.

6.5 Street segments (`engine/normalise/segments.py`)

Each shape is cut into **200 m** sections by distance along it. A section's value = mean level over the vehicles reporting there in the last **10 minutes**, each counted once at its latest reading; rounded to a level for colour. Overlapping routes are drawn worst-on-top.

6.6 Jobs, departures, service gap (`engine/demand/areas.py`)

- Jobs: zone's TZP24 2026 figure; grid cells take an area-weighted share.
- Departures: scheduled departures (all modes) at stops within **200 m**, **7:00–9:00** on a reference Tuesday (now **3 Nov 2026**).
- **Service gap** = percentile rank of jobs – percentile rank of departures, among areas with jobs (–1 to 1; positive = underserved relative to the rest of the footprint).

6.7 Opal 2016

Average weekday tap-ons + tap-offs, 7:00–9:00, 21–25 Nov 2016. Stations placed by name match ("Circular Quay, No. 3 Wharf" → "Circular Quay" + "Wharf"); a postcode's bus taps × share of its area inside the footprint, split evenly over today's bus stops there.

6.8 Pressure (`engine/demand/pressure.py`)

Inputs: traffic, crowding, service gap, Opal 2016, BOAM load — each a percentile rank across areas. Areas with no live or BOAM reading get the neutral **0.5** (dropping missing inputs put empty areas at the top of the ranking). Score = $\Sigma\, w_i \cdot r_i \div \Sigma\, w_i$, weights 0–3 (default 1). Jobs excluded (already in the service gap). Colours = fifths of the scores (averaged ranks bunch around 0.5).

6.9 BOAM sections (`engine/demand/boam.py`)

For consecutive stops a → b, weekdays only, per peak: **load** = occupancy band midpoint at a ÷ total capacity; **time lost per 100 m** = ((actual_b – sched_b) – (actual_a – sched_a)) ÷ section length × 100. Changes > 900 s discarded; ≥ 3 trips per section. Joined on **stop ID** (100% match; trip IDs don't match). Bands: load 15/30/45/60%; time lost 0/3/8/15 s per 100 m.

6.10 Our congestion measure — travel-time index (`engine/normalise/travel_time.py`)

- For each bus on a route shape: average speed over each stretch of **≥ 300 m**, stops and lights included.
- **Free-flow speed** per 200 m segment = **85th-percentile** night-time (22:00–05:00) speed from our recordings (`scripts/build_freeflow.py`; currently 4,712 segments from 7 nights, with 217 whole-route fallbacks). Segments without a baseline show grey.
- **TTI = free-flow speed ÷ current speed** (1.0 = as fast as at night), capped at 10.
- Levels on the model: **≤1.5 smooth, ≤2 slow, ≤3 congested, >3 severe**. In the analysis: **2.66 / 2.98 / 3.32**, fitted so our levels occur as often as TfNSW's (§8).

6.11 Colour bands summary

Layer	Bands	Kind
Jobs, Opal 2016	Fifths of areas with a value	Relative
Service gap	≤ –0.4, –0.15, ±0.15, 0.4, > 0.4	Fixed
Pressure	Fifths of scores	Relative
BOAM load	< 15, 30, 45, 60, > 60% of capacity	Absolute
BOAM time lost	≤ 0, 3, 8, 15, > 15 s/100 m	Absolute
Our traffic (model)	TTI 1.5 / 2 / 3	Absolute

---

7. Design decisions and justification

7.1 Why the teammates' calculations weren't reused

Flaw in the dashboard's congestion score	Why it misleads
Bus free-flow speed (30 km/h) applied to trains and Metro	Rail isn't in road traffic
Absolute delay treated as local	A bus late from Parramatta reads as congested in the CBD
Instantaneous speed	A bus at a stop or red light reads as slow
Bunching ignores the timetable	High-frequency routes run close by design
Grid cells average whatever is in them at that instant	No history; arbitrary boundaries

Their "demand" was jobs per zone — land use, not transit demand — so here it's labelled **jobs** and used inside the service gap. Their dashboard also committed a CARTO API key in plain text.

7.2 Major decisions

Decision	Why	Rejected alternative
Display state on the server, broadcast	Tablet, keyboard and projector can't disagree	State per page
Tablet controller, keyboard backup, PIN team panel, hotspot	Projector mirrors the screen; visitors need buttons; university Wi-Fi often blocks device-to-device	Keyboard only; visitors' phones
No basemap; clip to footprint	The model has the streets; anything outside lands on the table	Team's CARTO basemap
200 m route segments for moving things	Colour follows the streets on the model	Square grid
Zones and grid both, switchable	Judge on the real model which reads better	Pick blind
Service gap as a rank difference	Unit-free; a jobs/departures ratio explodes near zero departures	Ratio
Pressure: ranks, neutral fill, jobs excluded	Robust to outliers; no reward for missing data; no double counting	Min–max; drop missing inputs
Replay through the same engine on recorded time	One code path; correct windows at any speed	Pre-rendered video
25× time-lapse as showcase default	Shows congestion build and clear in minutes; independent of time of day and Wi-Fi	Live only
Record trimmed to the footprint	Statewide recording cost ~14 GB in weeks	Record everything

Decision	Why	Rejected alternative
Shared daily API budget; full rate only in peaks	Measured ~60k/day quota; 24/7 polling ran out at 7 pm	Fixed 15 s polling
Network errors retry ≤30 s	Wi-Fi drops cost half a minute, not five	Same backoff as 429s
BOAM joined on stop ID, weekdays only, absolute bands	Trip IDs don't match; bands stay comparable as days are added	Fifths
Step 6: travel-time index vs night free flow	Timetable-based delay was dominated by timetable padding (§8)	Delay vs timetable (method A)
Two threshold sets for our measure	Intuitive on the model; fair (frequency-matched) in the analysis	One set
Timetable refreshed 30 Oct	Old calendar ended 5 Nov, the day before the showcase	Keep the August bundle

8. Step 6 in detail: our congestion measure vs TfNSW

Method A (planned, dropped): time lost against the timetable per 100 m. On the 28 Oct AM peak the median bus *gained* 3–5 s per 100 m even where TfNSW reported severe congestion. Bus timetables are padded (buses run ahead and wait at timing points), so the measure mostly measured padding. Variants (100/300/500 m, stops in or out) didn't fix it. Worth one paragraph in the report as an informative negative result.

Method B (adopted): travel-time index against night-time free flow (§6.10). It needs no timetable.

Experiment to improve agreement (`scripts/experiment_congestion_collect.py`, `_evaluate.py`; thresholds fitted on Mon–Wed peaks, scored on Thu–Fri):

Baseline	Stretch	Has baseline	Fitted thresholds	κ per reading	κ per 10 min	Spearman per street, whole peak
Night	300 m	62%	2.66 / 2.98 / 3.32	0.009	0.015	**0.275**
Night	600 m	61%	2.42 / 2.70 / 3.01	0.003	0.012	0.104
Night	1000 m	62%	2.30 / 2.59 / 2.89	–0.009	0.003	0.181
Midday	300 m	73%	1.91 / 2.12 / 2.34	–0.022	–0.018	0.182
Midday	600 m	73%	1.70 / 1.87 / 2.05	–0.025	–0.016	0.038
Midday	1000 m	74%	1.62 / 1.80 / 2.01	–0.023	–0.013	0.156

Final run (`scripts/analyse_congestion.py`, 9 weekday peaks, 190,251 bus stretches, 62% with a baseline, analysis thresholds):

Scale	Result
Per reading (97,488 pairs)	Exact 63%, within one level 73%, **weighted κ 0.01**
Per segment per 10 min (83,634)	Exact 63%, κ 0.016
Per street, whole peak (1,184 streets)	**Spearman 0.24**
Median TTI by TfNSW level	Smooth 1.75 · Stop-and-go 2.03 · Congested 2.02 · Severe 1.93
vs BOAM time lost (2,207 segments)	Spearman 0.08

Interpretation for the report: per bus or per 10 minutes, our speed measure and TfNSW's label don't agree beyond chance; the 63% "exact" figure is mostly both calling ~77% of readings smooth (κ removes that). Averaged per street over a peak there is a weak but real relationship (ρ ≈ 0.24–0.28). Likely reasons: (1) the

night baseline has fewer passengers, so every peak street reads slower; (2) single-bus 300 m readings are noisy; (3) TfNSW's label is opaque and per vehicle — it may not be a speed measure of the street just driven; (4) BOAM time lost is timetable-relative (padding) and partly holiday days. **Conclusion: the two capture different things and only line up when averaged per street.**

Outputs: `data/analysis/congestion_agreement.json`, `congestion_disagreements.geojson` (337 segments differing by 2+ levels), `experiment_results.json`, `freeflow.json`.

---

9. Validation, testing and performance

- **193 automated tests** (`pytest`, ~2 s): static filter, map matching, adherence, aggregates, homography, footprint trimming, control state, segments/levels, replay (pacing, looping, coverage, seek, pause, writer choice), area maths, pressure, BOAM maths, API budget, travel-time tracker, agreement statistics, the teammate's stop tracker/history/trip updates/route focus, and the background map build. None cover the projector or tablet pages' JavaScript; those are checked by eye in a browser.
- **Live checks:** TfNSW field coverage (23 Oct): 164/269 buses with a traffic level; crowding on all buses and Metro, ~half of Sydney Trains, none for light rail/ferries. After the 30 Oct timetable refresh, all 324 live vehicles matched with route names.
- **Visual checks in Chrome** of the projector view (all layers) and the tablet (Display, Team, legend, timeline). Found and fixed: lines spilling past the model edge, a hidden row showing, recordings list missing after startup, nudge button layout.

Performance	Before	After
Live server CPU	~85%	~1.5%
Replay throughput, dense data	~11× real time	~96×
50× replay, measured	—	50.0×
Static layers build / cached	—	~5 s / 0.02 s
BOAM import	~400 MB/day	~2.7 MB/day
Polling within quota	Ran out at 7 pm	~57,000/day used, peaks at full rate

---

10. Operating it

10.1 Start / stop

```
python scripts/install_server_service.py                 # install the launchd job and start (done
launchctl kickstart -k gui/$(id -u)/com.pulsecbd.server  # restart, e.g. after a code change
python scripts/install_server_service.py --uninstall     # stop for good (plain `kill` only restart
```

The job starts the server at login, so after a reboot someone must log in. Startup ~15 s; area and BOAM layers finish in the background. Projector: `http://127.0.0.1:8765/` full-screen. Tablet: `http://<laptop address>:8765/control` (printed in the log).

10.2 Recording

- Scheduled: `python scripts/install_recording_schedule.py` (installed; `--uninstall` to remove). Wake schedule `scripts/wake_schedule.sh` was **not** run — not needed while sleep is disabled.

- Recordings: `data/recordings/{mode}/{feed_type}/{UTC hour}.{serve|rec}.ndjson.gz`. When both exist for an hour, replay reads the larger file (fixed 30 Oct; it used to prefer `rec`, which hid most of the Thu 29 Oct PM peak).

10.3 API budget (`engine/sources/budget.py`, config `api_budget`)

Shared daily count in `data/api_usage.json` (file-locked). Budget 57,000 (5% under the measured quota). Peaks (weekdays 06:55–09:05, 16:25–18:35) poll every 15 s; between them the interval stretches to keep the remaining peaks affordable; spent → one cycle per 600 s. The recorder idles while the server's heartbeat (`data/recordings/.server_polling`) is fresh.

10.4 Maintenance commands

```
python scripts/run_phase1.py --force-refresh                      # refresh the timetable (then res
python scripts/import_boam.py --delete-originals ~/Downloads/BOAM_*.txt
python scripts/export_boam.py                                     # data/boam/sections.csv
python scripts/build_freeflow.py                                  # rebuild night baseline (~5 min)
python scripts/analyse_congestion.py                              # step 6 analysis (~15 min)
python scripts/prepare_demand_data.py --poa-dir <ABS shapefile dir> # only if zones/postcodes change
python scripts/fetch_basemap.py [--cached]                        # background map from OpenStreetM
```

10.5 Weekly model session

1. Start the server; open both pages. 2. Team → verification overlay on; check corridors sit on the model's streets. 3. Small drift → nudge; large → crosshair calibration with ≥4 marks of known lat/lon (saving resets the nudge). 4. Try every layer on the model; note colours, line widths, zones vs grid.

10.6 Showcase-day checklist

- Laptop on power, hotspot on, tablet joined, `/control` open, PIN unlocked once.
- Projector view full-screen, status line off.
- Verification overlay check, nudge if needed.
- Default: 25× replay of the best weekday peak; live if Wi-Fi and API are healthy; **return to Live** afterwards.
- Team → System health shows fresh feeds before going live.

---

11. Repository and branches

Branch	Where	State
`w12merge`	local + GitHub	**Current, and what the Mac runs.** `yosephW12` merged with the teammate's `integration` work (merge `3ab2a8a`, 30 Oct), plus later work: stop visits recorded only while live, the server's launchd job, the background map.
`yosephW12`	local + GitHub	The owner's work through 30 Oct, before the merge (`3bfd432`).
`integration`	GitHub `67d4d9c`	The teammate's 11 commits of 29–30 Oct (stop history, delay-reading fixes, route filter, arrows, Windows support, their individual report) on top of the owner's work to 26 Oct. Fully contained in `w12merge`.
`test`	local + GitHub	Owner's prototype as of 28 Aug

Branch	Where	State
`main`, `mid-project-demo`, `test-pipline`, `Mid-project-demo-A-edits`	GitHub	Teammates' template and dashboard work

Which branch is the showcase build still has to be agreed with the teammate (§16); `w12merge` is the only one with both sides' work.

Never push to `integration` **without explicit instruction** (owner's 30 Oct instruction).

Repository clean-up before submission: the teammate committed personal `.claude/settings.json` and `.vscode/settings.json`, ~55 MB of PDFs and a spreadsheet in `template-repository-main/New folder/` (not in Git LFS), and their report at the repo root.

Security note: a teammate's TfNSW API key is committed in `template-repository-main/src/file.env` (commit `d85b5e1`, private repo). It should be regenerated. The owner's key is only in `.env` (gitignored) and appears in no commit (checked before each push).

---

12. State of the Mac (as of 4 Nov, 8 pm)

Item	State
Server	launchd job `com.pulsecbd.server` (since 3 Nov), running `w12merge`, live mode, logging to `data/recordings/server.log`
launchd recorder	Installed, idle between peaks, stands aside while the server polls
Sleep	Disabled (`SleepDisabled 1`) — undo after 6 Nov: `sudo pmset -a disablesleep 0`. The Mac must stay **on power** (it was found on battery on 3 Nov)
API usage	~57,000 every full day 26 Oct – 2 Nov · 3 Nov 39,663 (server down 11:30–18:33) · 4 Nov 46,023 by 8 pm
Disk	73 GB free. `data/recordings` 26 GB, `data/history` (stop visits) 112 MB, `data/basemap` 16 MB
Node.js	Installed via Homebrew (needed only for syntax checks / skills)
Claude skills installed	grill-me, grilling, grill-with-docs, loop-me, domain-modeling, handoff, wait-what (others removed)

Weekday peak recordings so far (coverage of expected bus updates): Mon 26 Oct AM 100%, PM 100% · Tue 27 AM 100%, PM 86% (Wi-Fi drop ~5 pm) · Wed 28 AM 100%, PM 100% · Thu 29 AM 100%, PM 99% · Fri 30 AM 99%, PM 100% · Mon 2 Nov AM 100%, PM 98% · Tue 3 AM 100%, **PM lost** (Mac shut down; see `DECISIONS.md`, 3 Nov) · Wed 4 AM 100%, PM 99%. Weekends 24–25 Oct and 31 Oct – 1 Nov (66%: slower polling outside weekday peaks), 6 Sep and 22 Aug are also replayable.

Known outages: Wi-Fi drops 23 Oct from ~4:31 pm (lost most of that PM peak), 25 Oct afternoon, 27 Oct 5:00–5:20 pm and 7:40–10:42 pm; TfNSW throttled the key 2–4 pm on 28 Oct (not our quota); the Mac shut down after 11:30 am on 3 Nov and the server stayed down until 6:33 pm.

---

13. Limitations and caveats (state these in the final report)

Data - TfNSW's traffic label is undocumented; crowding has four useful categories and is almost always "seats available" off-peak; light rail and ferries report neither; Sydney Trains traffic is always unknown. - BOAM

"actual" times are GTFS-R predictions since Mar 2026; non-tappers (notably students) aren't counted; occupancy is assumed to be load departing each stop. - Opal 2016 is ten years old and pre-Metro; bus taps are only by postcode. - Recordings have gaps when Wi-Fi drops.

Method - Service gap and pressure are **relative to the footprint**, not planning standards. - Departures count once per stop within reach — service reachable, not distinct trips. - Overlapping route segments are drawn worst-on-top; no physical street network. - Adherence thresholds are simplified defaults (§6.2); early running counts as on time. - Our TTI uses a night baseline (fewer passengers → peak reads slower everywhere); 38% of peak readings have no baseline; agreement with TfNSW is weak (§8).

Setup - **Nothing tested on the physical model yet**; calibration only with synthetic points; projector resolution/aspect unknown. - PIN is browser-only; server listens on all interfaces (use the hotspot). - Single laptop; replay depends on its disk. - Branch divergence with the teammate's `integration` work (§11).

---

14. Remaining work (prioritised)

Before the showcase (by 5 Nov) 1. ~~**Merge `yosephW12` with the teammate's `integration` work**~~ — done 30 Oct (`w12merge`). Still to do: **agree with the teammate** that `w12merge` is the showcase build, and that ~70% of Sydney Trains now show grey. 2. **Test on the physical model** at the next weekly session: calibrate, check every layer, zones vs grid, colours on white card. Fixes allowed after the freeze. 3. ~~**Check the Thu 29 Oct PM peak** (50% coverage)~~ — done 30 Oct: a replay bug, not missing data (see `DECISIONS.md`). 4. **Keep recording** through Thu 5 Nov; keep the Mac plugged in and on Wi-Fi during 06:55–09:05 and 16:25–18:35. 5. **Rebuild the free-flow baseline** and **rerun the step 6 analysis** once more nights and peaks are in (`build_freeflow.py` → restart → `analyse_congestion.py`). 6. **Report figures:** service gap, pressure, BOAM load and time lost, the step 6 tables and disagreement map. 7. **Pick the showcase replay** (best complete weekday peak). 8. **Full rehearsal** with the tablet on the hotspot. **Test the networking first:** a Mac can't host a hotspot and join Wi-Fi at once, so live mode needs Ethernet or USB tethering, or a phone hotspot both devices join. 9. Ask the teammate to **regenerate the committed API key**. 10. **Back up the best 3–4 peaks** to an external drive (replays depend on this one disk). 11. **Say on the projection when it's a replay** (status line, or say it out loud): the default is a 25× replay of a past peak.

Optional / undecided - Exclude the 3 holiday BOAM days from averages (they barely change results — loads match to ~1 percentage point). - Wi-Fi stall notification on the Mac (offered, not built). - Add an "early" adherence state; cite official TfNSW on-time definitions. - Move "Traffic (our measure)" to the team panel (it disagrees with TfNSW's layer and reads "slow" almost everywhere at peak). - Speed up `/stop_history` (rescans the whole history; ~13 s at 586k visits). - Check the Full background map's labels are readable on the model.

After the showcase: `sudo pmset -a disablesleep 0`; `python scripts/install_server_service.py --uninstall`; `python scripts/install_recording_schedule.py --uninstall`.

---

15. Suggestions and room for improvement

1. **Guided demo mode:** one button plays a scripted tour (AM peak 25× → traffic → service gap → pressure) with captions on the tablet.
2. **Trim the visitor tab:** move BOAM, grid toggle and our-measure behind "More".
3. **Step 6 improvements** (analysis only): baseline by time of day from weekday shoulder hours; per-route rather than per-segment comparison; account for dwell by excluding stops on both sides.

4. **Validate the service gap** against an accessibility measure (e.g. jobs reachable in 30 minutes) for sample zones; sensitivity tests (walk distance 400 m, each pressure weight at 0).
5. **Deduplicate shared streets** by snapping segments to a street network (OpenStreetMap).
6. **BOAM passenger-kilometres** rather than mean load; term-time only.
7. **Speed up startup** by caching route corridors like the area layers.
8. **Touch markers on the model** at stations (original brief, out of scope this term).

---

16. Open decisions waiting on the owner

1. Exclude the holiday BOAM days from averages?
2. Build the Wi-Fi stall notification?
3. Confirm `w12merge` as the showcase build, with the teammate.
4. Move "Traffic (our measure)" off the visitor tab?
5. Speed up the teammate's `/stop_history` (their code)?

---

17. Timeline of the work

Date	What happened
Aug	Prototype phases 1–6 built (engine, renderer, calibration, stats panel) on `test`
22–23 Aug	Proposal and timeline documents
25 Sep	Design "grilling" session; plan, decisions and build order agreed
23 Oct	Steps 1–3 built (footprint, tablet controller, recording, traffic/crowding segments, replay); rate-limit and map-matching performance fixed; first scheduled PM peak lost to Wi-Fi
23 Oct	Steps 4–5 (area layers, pressure); BOAM imported (5 days) and used three ways; branch `integration` pushed
24–25 Oct	Server run continuously; quota exhausted 7 pm Sun → quota measured
26 Oct	Quota-aware polling, network retry ≤30 s; first complete weekday peaks
29–30 Oct	Teammate pushes 9 commits to `integration`
30 Oct	Replay pause/seek; timetable refreshed; step 6 (method B, experiment, final analysis); 12 more BOAM days (term time); work pushed to new branch `yosephW12`; replay writer fix (Thu 29 PM 50% → 99%); merged with `integration` as `w12merge`; stop visits live-only
3 Nov	Mac shut down after 11:30, server down until 18:33, PM peak lost; server moved to a launchd job that restarts it
4 Nov	Optional background map: Plain / Detailed / Full (OpenStreetMap, offline, with labels)

---

18. Key files

Area	Files
Config	`config/config.yaml` (footprint, feeds, thresholds, api_budget, derived_congestion, server, controller PIN)
Live engine	`engine/sources/live.py`, `budget.py`, `recorder.py`, `footprint.py`, `replay.py`
Normalise	`engine/normalise/build.py`, `enrich.py`, `map_match.py`, `levels.py`, `segments.py`, `travel_time.py`, `adherence.py`, `aggregate.py`

Area	Files
Static layers	`engine/demand/areas.py`, `boam.py`, `pressure.py`
Server	`engine/publish/server.py`, `feed.py`, `control.py`, `calibration.py`, `route_focus.py`
Stop history (teammate)	`engine/history/stop_tracker.py`, `store.py`; `engine/normalise/trip_updates.py` (delay rules)
Pages	`renderer/index.html` (projector), `renderer/controller.html` (tablet)
Scripts	`run_publish.py`, `run_record.py`, `install_server_service.py`, `install_recording_schedule.py`, `fetch_basemap.py`, `run_phase1.py`, `prepare_demand_data.py`, `import_boam.py`, `export_boam.py`, `build_freeflow.py`, `analyse_congestion.py`, `experiment_congestion_collect.py`, `experiment_congestion_evaluate.py`
Data (gitignored)	`data/static_cache/`, `data/recordings/`, `data/demand/`, `data/opal/`, `data/boam/`, `data/analysis/`, `data/history/` (stop visits), `data/basemap/`, `data/api_usage.json`, `data/projection_nudge.json`
Docs	`README.md`, `docs/DECISIONS.md`, `docs/PROJECT_REPORT.md`